Introduction to Computer Systems

Lecture 6: Concurrency and Synchronization (1)

Instructors: Wei Xu, Mingyu Gao

Teaching Assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

April 7, 2026

Contents

1 Introduction **3**

2 Clarification of the Last Lecture: Shell Truth Values and Build Automation **3**

2.1 Shell exit status as control flow 3

3 From Processes to Threads **4**

3.1 Processes as execution contexts 4

3.2 Threads as independent execution sequences 5

3.3 Processes versus threads 5

3.4 Why allow cooperating threads? 5

3.5 Shared memory and message passing 6

3.6 Web servers, thread pools, and event-driven systems 6

4 The Challenge of Shared State **8**

4.1 Interleaving is the real adversary 8

4.2 Race conditions, non-determinism, and debugging 8

4.3 Therac-25: correctness failures in the real world 9

5 Foundations of Synchronization **9**

5.1 Atomic operations 9

5.2 Critical sections and mutual exclusion 10

5.3 Locks and waiting 10

5.4 Granularity: coarse versus fine 10

6 The "Too Much Milk" Problem **12**

6.1 Problem statement and correctness properties 12

6.2 Naive note-based solutions and why they fail 12

6.3 A correct but unsatisfactory busy-wait solution 12

6.4 What the problem teaches 13

6.5 Locks make the intent explicit 13

7 Semaphores **13**

7.1 Definition and intuition 13

7.2 Binary semaphore versus lock 14

7.3 Semaphores for scheduling constraints . . . 15
7.4 Producer-consumer and the bounded buffer . . . 15
7.5 Why semaphores are a major step forward . . . 16

8 Monitors and Condition Variables **17**
8.1 Why semaphores are not the final abstraction . . . 17
8.2 Condition variables . . . 17
8.3 A monitor-based synchronized queue . . . 17
8.4 Hoare monitors and Mesa monitors . . . 18
8.5 Why Mesa monitors require `while` . . . 19

9 System Design Lessons and Historical Notes **20**

10 Summary **21**

11 Quizzes & Exercises **21**
11.1 Short-answer questions . . . 21
11.2 Essay prompts . . . 22
11.3 Reasoning exercises . . . 22

1 Introduction

Concurrency is one of the central ideas of computer systems. Operating systems multiplex processors across many activities, web servers handle many clients at once, user interfaces must remain responsive while background work continues, and modern multicore hardware invites programs to decompose work into parallel pieces. Yet the very abstraction that makes these systems powerful also makes them hard to reason about. Once multiple threads of control can observe and update shared state, the behavior of a program depends not only on the source code, but also on *when* operations occur relative to one another.

This lecture begins with a short warm-up on shell command composition and build systems. That material is not accidental. The shell already teaches a systems way of thinking: commands have exit status, complex workflows are built by composing smaller parts, and a build system is essentially a dependency manager for many generated artifacts. The same mindset reappears in concurrency. We will again ask what depends on what, which actions must happen before others, and how a system can enforce those constraints.

The main technical arc of the lecture moves from processes and threads, to race conditions and synchronization, to the classic "too much milk" problem, and finally to two major synchronization abstractions: semaphores and monitors. Along the way we will study why naive solutions fail, why some correct solutions are still unsatisfactory, and why higher-level abstractions are indispensable in real software.

Deep Dive

A useful mental model for the entire lecture is this: concurrency introduces *more possible executions* than a sequential program has. Synchronization is the art of ruling out the executions that would violate correctness while preserving as much performance and modularity as possible.

2 Clarification of the Last Lecture: Shell Truth Values and Build Automation

2.1 Shell exit status as control flow

In C and C++, the integer value `0` is conventionally interpreted as false and nonzero values as true in conditional expressions. The Unix shell uses a different convention for command execution: an exit status of `0` means *success*. In shell conditionals, success is what matters. This is why the shell operators `&&` and `||` can feel reversed to beginners even though their short-circuit behavior is not reversed at all.

Listing 1: Shell control flow based on command success or failure.

```
ping -c 1 google.com && wget https://example.com/file.zip
make || echo "build failed"
cmd1 ; cmd2
```

The meaning of these operators is straightforward once exit status is understood:

- `cmd1 && cmd2`: run `cmd2` only if `cmd1` succeeds.
- `cmd1 || cmd2`: run `cmd2` only if `cmd1` fails.

- `cmd1 ; cmd2`: run both commands sequentially regardless of success.

This seemingly small convention matters because it turns the shell into a lightweight programming language for system administration and build automation. The shell is not merely a launcher of commands; it is an interface in which operating-system return codes become the basis of branching, fallback, and composition.

3 From Processes to Threads

3.1 Processes as execution contexts

A process is more than a program text stored on disk. At run time, the operating system must maintain an *execution context*: the address space visible to the process, the current values of architectural registers, the program counter, stack state, and other kernel bookkeeping required to suspend and resume execution.

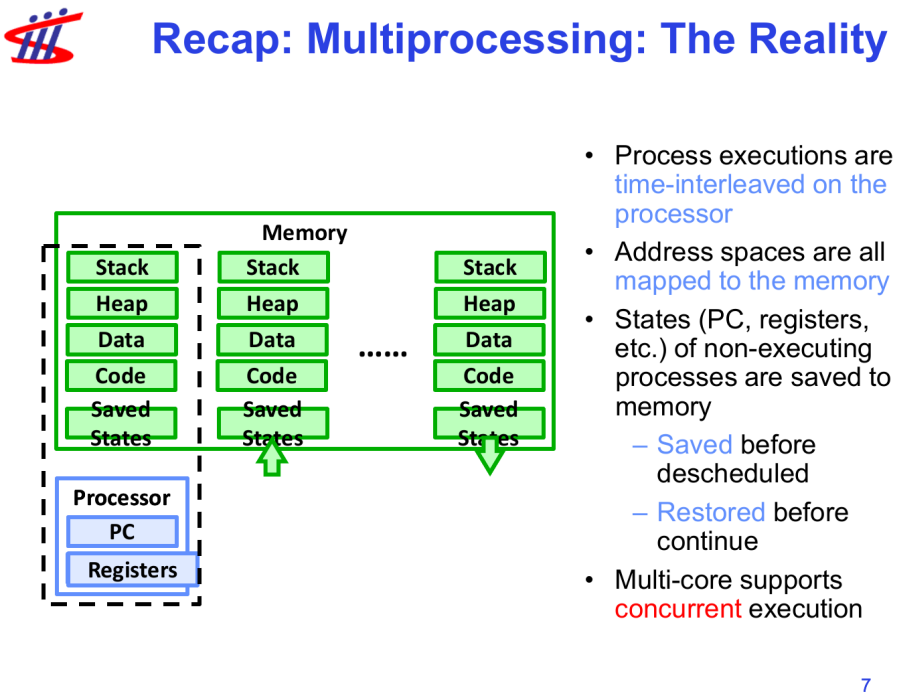


Figure 1: The basic reality of multiprocessing on a single processor. Multiple processes occupy memory simultaneously, each with its own code, data, heap, stack, and saved architectural state, but only one process executes on the processor at a given instant. Context switching saves the current state of one process and restores the state of another.

On a single core, processes are typically *time-interleaved*. The illusion of many simultaneously running programs is created by rapid context switching:

1. the kernel saves the state of the currently running process;
2. the scheduler chooses another ready process;
3. the kernel restores that process's state and resumes it.

On multicore hardware, several processes may run truly concurrently. Nevertheless, the conceptual lesson is the same: the kernel owns the schedule, and software must be correct under the interleavings that schedule permits.

3.2 Threads as independent execution sequences

A thread is a single sequence of control within a process. Each thread has its own program counter, register set, stack, and scheduling state, but threads within the same process share the code, data segment, heap, open files, and address space of the process as a whole.

Definition

A **thread** is an independently schedulable execution context that owns its own CPU state and stack, while typically sharing the process address space and resources with sibling threads.

Because threads share an address space, switching between two threads in the same process is often cheaper than switching between processes. The kernel usually need not install a new address space, and communication through shared variables can be much cheaper than interprocess communication. Yet thread switching is not free: scheduler overhead, cache disruption, and lock contention can still dominate performance in poorly designed systems.

3.3 Processes versus threads

Property	Process	Thread
Address space	Separate virtual address space	Shared with sibling threads in same process
Execution state	Own PC, registers, stack, kernel metadata	Own PC, registers, stack, scheduling state
Communication	Pipes, sockets, shared memory, message passing	Shared variables, queues, condition variables, locks
Isolation	Stronger fault containment and protection boundary	Weaker isolation because memory is shared
Switching cost	Usually higher	Usually lower, but still nontrivial
Typical use	Separate programs or strong isolation domains	Fine-grained concurrency within one program

Table 1: Threads trade stronger isolation for cheaper sharing and lower context-switch cost.

3.4 Why allow cooperating threads?

Three major reasons for multithreading:

Resource sharing. Many activities must access one logical resource: a single file cache in a web server, one bank balance visible at many ATMs, or a robot controller coordinating sensors and actuators.

Speedup. Threads allow I/O and computation to overlap. When one thread blocks on disk, network, or user input, another thread can continue using the CPU. On multicore machines, threads also enable actual parallel execution.

Modularity. Threads let programmers decompose a large task into simpler sequential pieces. A compiler pipeline, a server that separates request acceptance from request handling, or a UI that separates rendering from background work can all be easier to write as interacting threads than as a single giant state machine.

A particularly vivid example is user-interface responsiveness. If a program performs a slow disk read or network fetch on the same control path that is responsible for drawing the screen, the interface may appear frozen. Users then retry actions, often creating more work. Concurrency helps maintain the illusion that the system is continuously responsive even when some subsystems are waiting on slow devices.

3.5 Shared memory and message passing

When multiple activities collaborate, they need a communication mechanism. Two broad models dominate:

- **Shared-memory mapping:** cooperating threads or processes read and write common memory locations. Communication is low latency but requires careful synchronization.
- **Message passing:** computations exchange information with `send()` and `receive()` operations, often through kernel queues or network channels. This model can work across machines and provides a clearer separation between participants.

The trade-off is classic. Shared memory is often fast and natural within one process, but it exposes programs to races. Message passing narrows the interface between participants, but may impose copying or protocol overhead.

3.6 Web servers, thread pools, and event-driven systems

A canonical example of threads in systems programming is a web server. A simple multithreaded server accepts a new client connection, creates a thread, and lets that thread process the request from beginning to end. This style is appealing because each thread can be programmed in a direct sequential style: read request, compute response, send bytes, terminate.

However, unbounded thread creation is dangerous. Every thread needs memory for its stack and bookkeeping; too many threads increase scheduler overhead, memory pressure, and context-switch frequency. Past a certain point, more threads reduce throughput rather than improve it. This phenomenon is sometimes described as a *throughput sink*.

A thread pool addresses this problem by bounding concurrency. The program preallocates a fixed number of worker threads, keeps incoming work in a queue, and lets idle workers sleep when no request is available.

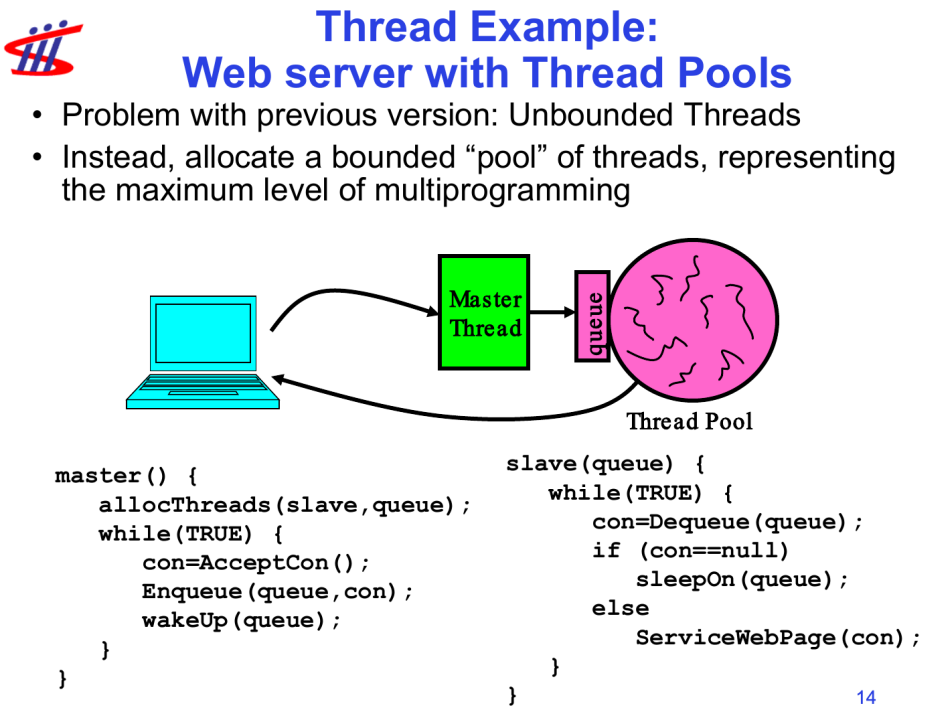


Figure 2: A bounded thread-pool web server architecture. A master thread accepts incoming connections, places work items into a shared queue, and wakes a worker from a preallocated pool. The figure captures the systems goal of preventing unbounded thread creation while maintaining concurrency and service throughput.

Algorithm 1: Bounded Thread-Pool Web Server

Input: Listening socket L, work queue Q, worker count N
Output: Incoming requests are serviced without unbounded thread creation

allocate N worker threads attached to Q;
foreach *worker thread* **do**
  while *true* **do**
    wait until Q is nonempty;
    remove one connection from Q;
    service the request to completion;
while *true* **do**
  accept a connection on L;
  enqueue the connection into Q;
  wake one sleeping worker, if any;
return result;

The transcript also contrasts threaded servers with event-driven servers such as Nginx. An event-driven design can be extremely efficient because it avoids per-request thread overhead and expresses I/O as a state machine driven by readiness notifications. The cost is programming complexity: instead of writing one natural sequential request handler, the programmer manually manages state transitions. This is a recurring theme in systems design: the most efficient low-level mechanism is not always the easiest to reason about or maintain.

4 The Challenge of Shared State

4.1 Interleaving is the real adversary

Threads are independently scheduled by the operating system. They may start, stop, block, wake, and resume at times that the programmer does not explicitly choose. If threads work on disjoint data, this is often harmless. If they access the same variable or data structure, the result can depend on the order of reads and writes.

Consider the following simplified shared-variable example.

Listing 2: A tiny race on shared state.

```
int x = 1;
int y = 12;

/* Thread A */
x = y + 1;

/* Thread B */
y = 2;
y = y * 2;
```

Possible outcomes include:

- $x = 13$**:** thread A reads the original value $y = 12$ before thread B modifies it.
- $x = 5$**:** thread B runs to completion first, so thread A later reads $y = 4$.
- $x = 3$**:** thread B stores $y = 2$, thread A reads that intermediate value, and only afterward does thread B complete the multiplication.

The important lesson is that concurrent programs must be correct for *all* allowed interleavings, not merely for the interleavings that occur frequently on one machine in one run.

Deep Dive

In the C and C++ memory model, an unsynchronized conflicting access to the same memory location is a *data race*, and a program with a data race has undefined behavior. This is stronger than merely saying that outcomes may vary: the language gives the implementation wide latitude once the program violates the synchronization rules.

4.2 Race conditions, non-determinism, and debugging

A **race condition** occurs when correctness depends on the relative timing of concurrent operations. Such bugs are especially hard to diagnose because they are often:

- **non-deterministic:** the bug does not appear on every run;
- **non-reproducible:** small changes in workload, compiler, machine, or timing alter behavior;
- **observer-sensitive:** adding logging or debugging output can change the schedule enough to hide the bug.

This last phenomenon gives rise to the term *Heisenbug*. The act of observing the system perturbs the system. A strategically placed `printf` can alter lock timing, memory layout, or I/O behavior and make a race disappear temporarily without actually fixing it.

4.3 Therac-25: correctness failures in the real world

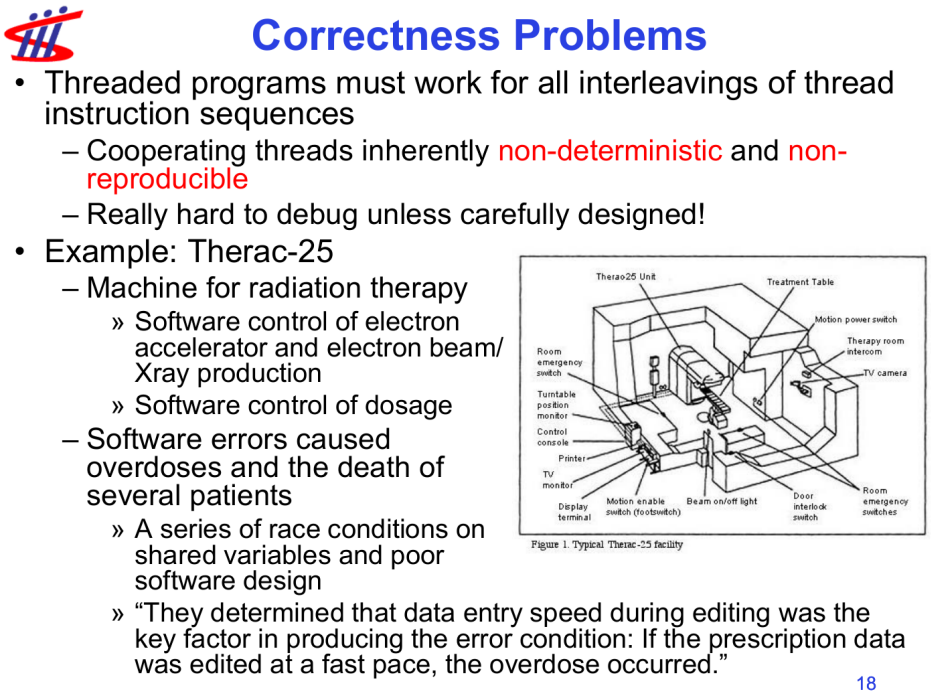


Figure 3: The Therac-25 case study, as a warning that concurrency errors are not merely academic. The case connects nondeterministic interleavings, race conditions on shared variables, and poor software engineering practice to catastrophic consequences in a safety-critical radiation-therapy system.

Therac-25 is a classic systems case study because it demonstrates that timing errors can have physical consequences. Software controlled dosage and beam configuration in a radiation-therapy machine, and under particular timing conditions the machine delivered dangerous overdoses. A key lesson from postmortem analysis is that rare interleavings are still real interleavings. A bug that appears only when an operator edits data quickly is still a correctness bug; it is merely one that the test regime failed to expose.

The case also highlights an engineering principle broader than concurrency. When synchronization design is flawed from the outset, patching around symptoms can produce an opaque and fragile system. Correctness in concurrent software is largely a *design-time* property, not something that can reliably be retrofitted after the program grows large.

5 Foundations of Synchronization

5.1 Atomic operations

To reason about a concurrent program, one must first know the atomic operations from which larger coordination mechanisms are built.

Definition

An **atomic operation** is an operation that appears indivisible: it either completes entirely or does not occur at all, and no other thread can observe or modify the relevant state halfway through the operation.

On most conventional machines, a single aligned machine-word load or store is atomic. Larger operations often are not. Updating a structure, a long integer on an older machine, or an array element sequence may require multiple lower-level actions, each of which can be interleaved with other threads.

Atomicity matters because synchronization cannot be built from nothing. If every observation and update could itself be torn apart arbitrarily, no higher-level notion of coordination would be reliable.

5.2 Critical sections and mutual exclusion

A **critical section** is a piece of code that should be executed by at most one thread at a time. The property that guarantees this is **mutual exclusion**. The two phrases describe the same core idea from different angles: one emphasizes the protected region, the other emphasizes the exclusion mechanism.

Definition

Synchronization is the use of atomic operations and higher-level coordination primitives to ensure that concurrently executing threads satisfy required safety and liveness properties.

5.3 Locks and waiting

A **lock** is the standard abstraction for mutual exclusion. A thread acquires the lock before entering a critical section and releases it when leaving. If another thread already holds the lock, the arriving thread waits.

There is an important conceptual point: *all synchronization involves waiting.* Sometimes the waiting is visible, as when a thread blocks on a mutex. Sometimes it is hidden by hardware, the kernel, or a runtime. But synchronization always constrains progress to preserve correctness.

5.4 Granularity: coarse versus fine

Choosing the granularity of synchronization is a classic engineering trade-off.

Examples arise everywhere: database table locks versus row locks, a single giant kernel lock versus many subsystem locks, or a refrigerator lock that blocks all access versus a narrow protocol that only serializes milk purchasing.

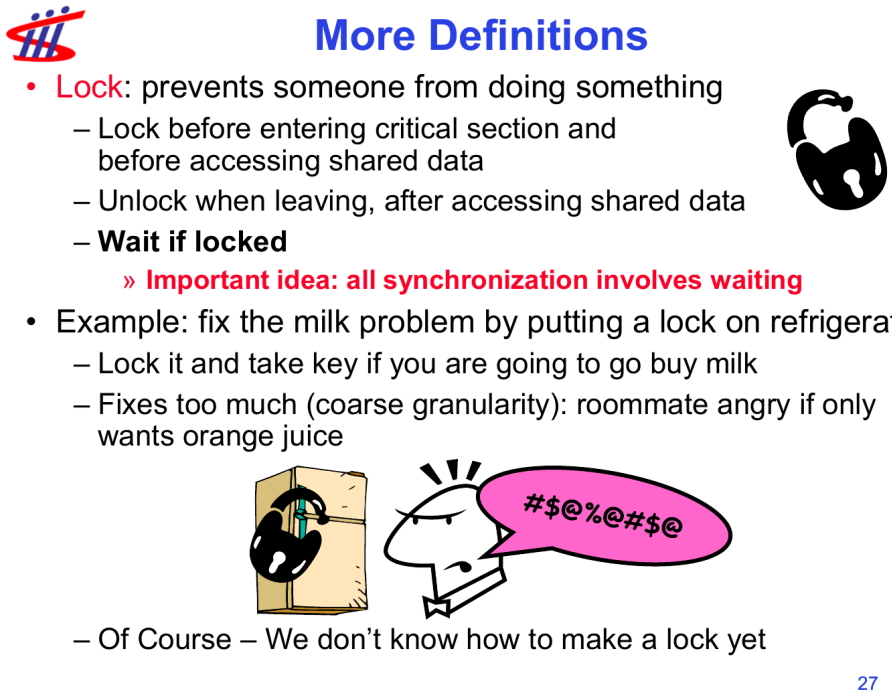


Figure 4: Locking the refrigerator prevents another participant from simultaneously performing the milk-buying protocol, but it also illustrates the cost of overly coarse synchronization: unrelated actions, such as merely checking for orange juice, are blocked as well.

Approach	Advantage	Cost
Coarse-grain locking	Simpler reasoning, fewer locks, smaller correctness surface	Reduced concurrency; unrelated operations may block each other
Fine-grain locking	Higher parallelism and better throughput when contention is low	More complex code; more opportunities for deadlock, races, and lock-order mistakes

Table 2: Synchronization granularity trades simplicity against performance.

6 The "Too Much Milk" Problem

6.1 Problem statement and correctness properties

The "too much milk" problem translates a real-world coordination issue into a synchronization problem. Two roommates independently inspect the refrigerator. If they see no milk, one of them should buy milk. The challenge is to guarantee:

- **Safety:** never buy more than once.
- **Liveness:** if the refrigerator is empty, eventually someone buys milk.

These are the two correctness properties: "never more than one person buys" and "someone buys if milk is needed."

6.2 Naive note-based solutions and why they fail

A first attempt is to leave a note on the refrigerator before buying milk and remove the note afterward:

Listing 3: A naive note-based protocol.

```
if (noMilk) {
    if (noNote) {
        leaveNote();
        buyMilk();
        removeNote();
    }
}
```

This fails because checking the note and placing the note are separate actions. One roommate can be interrupted after checking `noNote` but before leaving the note; the other roommate then observes the same state and also proceeds. The resulting bug is intermittent, which makes it harder to diagnose than a scheme that fails every time.

Reversing the order does not help:

Listing 4: Posting the note first is also incorrect.

```
leaveNote();
if (noMilk) {
    if (noNote) {
        buyMilk();
    }
}
removeNote();
```

Now nobody may buy milk, because each participant can be blocked by the very note they posted. A further variant using two labeled notes also fails: both roommates may leave their own notes, each observe the other's note, and both decide that the other will handle the task.

6.3 A correct but unsatisfactory busy-wait solution

Actually there is a two-note solution that can be made correct using busy waiting. One participant waits while the other participant's note is present, then rechecks the milk condition. The logic

can be shown to satisfy the desired safety property, but it is unsatisfactory for several reasons:

- it is asymmetrical—roommate A and roommate B run different protocols;
- it scales poorly beyond two participants;
- while waiting, a participant repeatedly polls the state instead of sleeping, wasting CPU time in the systems analogue of standing in front of the refrigerator doing nothing useful.

Deep Dive

Busy waiting is sometimes appropriate in very low-level contexts, such as short spin-wait loops in kernel code or on multiprocessors where sleeping would be more expensive than briefly spinning. At the level of general application synchronization, however, busy waiting is usually a sign that the abstraction is too low-level for the problem.

6.4 What the problem teaches

The milk problem is valuable not because anyone really synchronizes roommates with notes, but because it forces precise reasoning about *what must be true* rather than *what usually happens*. Once the desired behavior is formalized, it becomes clear why ad hoc communication schemes are fragile and why programmers seek stronger primitives than plain loads and stores.

6.5 Locks make the intent explicit

If a correct lock abstraction already exists, the milk problem becomes easy:

Listing 5: The milk problem with an ideal lock abstraction.

```
milkLock.Acquire();
if (noMilk) {
    buyMilk();
}
milkLock.Release();
```

The code between `Acquire()` and `Release()` is a critical section. The point here is that such a solution is simple only because the lock itself embodies a nontrivial correctness guarantee. The real work has moved into the implementation of the synchronization primitive.

7 Semaphores

7.1 Definition and intuition

The semaphore is one of the oldest and most important synchronization primitives.

Definition

A **semaphore** is a nonnegative integer together with two atomic operations:

- `P()` waits until the value is positive, then decrements it by one;
- `V()` increments the value by one and wakes a waiting thread, if any.

Historically, `P` comes from the Dutch *proberen* (to test) and `V` from *verhogen* (to increment).

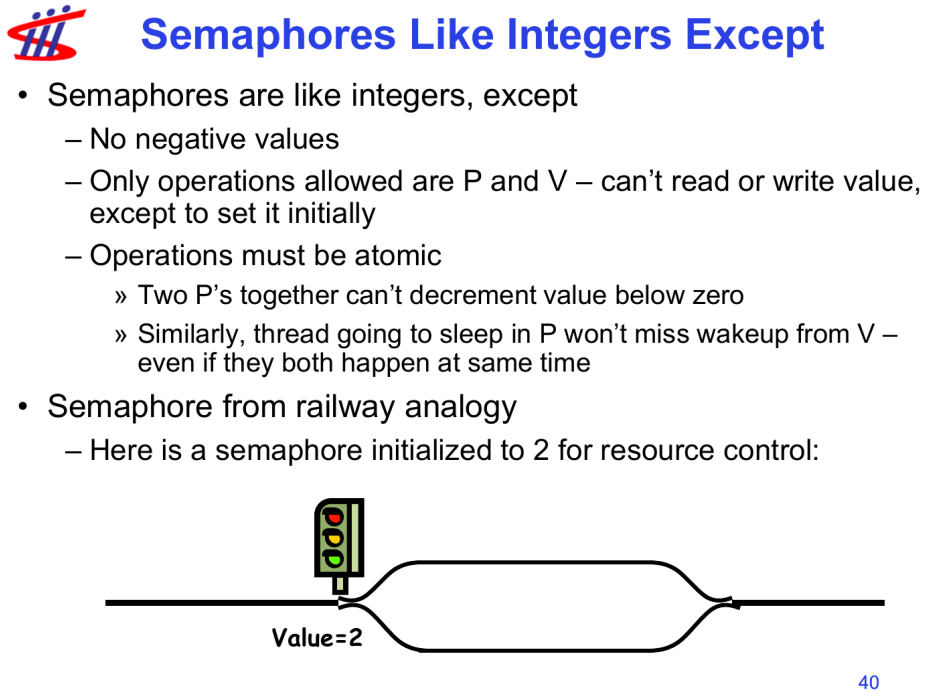


Figure 5: The railway analogy for counting semaphores. Initializing a semaphore to 2 models a resource with two available slots. Each `P()` operation consumes one slot; once the count reaches zero, further arrivals must wait until a `V()` operation returns capacity.

A semaphore resembles an integer but with crucial restrictions:

- it never becomes negative;
- arbitrary read-modify-write access is not permitted;
- only the atomic operations `P()` and `V()` manipulate it after initialization.

7.2 Binary semaphore versus lock

A semaphore initialized to 1 is often called a **binary semaphore**. It can enforce mutual exclusion:

Listing 6: Binary semaphore used for mutual exclusion.

```
mutex.P();
/* critical section */
mutex.V();
```

Although this resembles a lock, the two abstractions are not identical. Locks carry an ownership convention: the thread that acquires a lock is the thread that should release it. Semaphores are more general counters. If a programmer accidentally performs extra `V()` operations, the semaphore count can grow beyond 1 and the intended mutual exclusion disappears. This is one reason why higher-level lock abstractions remain useful even though a binary semaphore can emulate them.

7.3 Semaphores for scheduling constraints

Semaphores are equally useful for expressing ordering constraints. Suppose a main thread must wait for a child thread to terminate. A semaphore initialized to 0 naturally captures this dependency:

- the waiting thread calls `P()` and sleeps because the value is 0;
- the finishing thread calls `V()` and thereby wakes the waiter.

This use of semaphores is conceptually different from mutual exclusion. The semaphore is not protecting a critical section; it is recording that an event has occurred.

7.4 Producer-consumer and the bounded buffer

The producer-consumer problem introduces a finite queue shared by producers and consumers. Producers place items into the buffer; consumers remove them. The buffer decouples their rates so that the two sides need not run in lockstep.

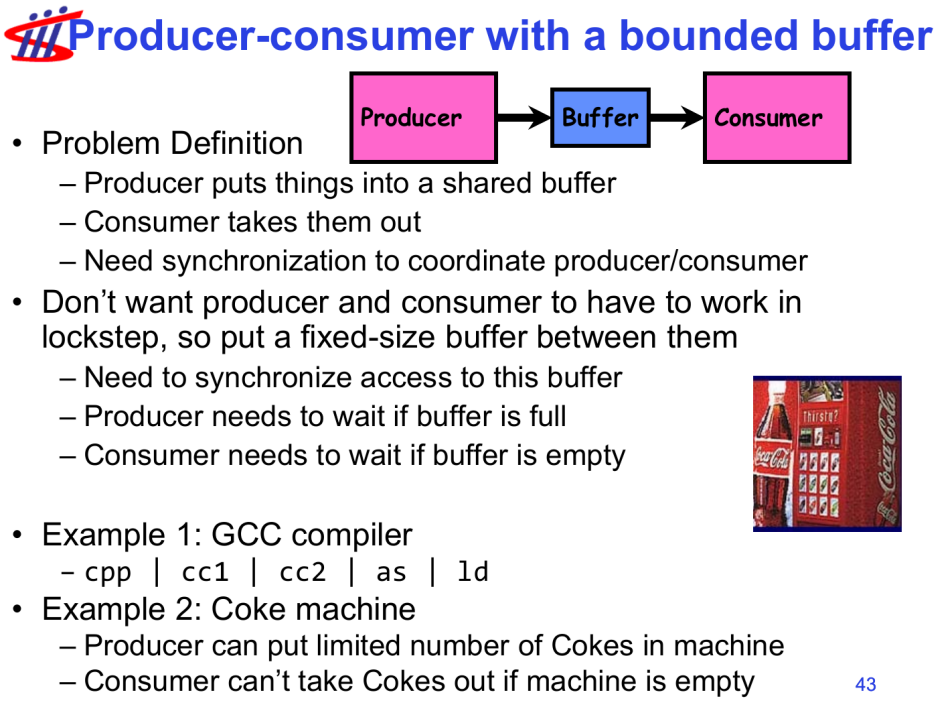


Figure 6: The bounded-buffer abstraction. Correctness requires both scheduling constraints (empty/full behavior) and mutual exclusion for buffer updates.

Three constraints appear immediately:

1. a consumer must wait if the buffer is empty;
2. a producer must wait if the buffer is full;
3. only one thread at a time should modify the shared queue internals.

A standard design rule is to use a separate semaphore for each logically distinct constraint:

- `fullSlots`: how many items are available for consumers;
- `emptySlots`: how many free positions remain for producers;
- `mutex`: whether the queue internals are currently being manipulated.

Algorithm 2: Producer Operation for a Bounded Buffer

Input: Item x, queue Q, semaphores $emptySlots$, $fullSlots$, and $mutex$
Output: x is inserted once buffer space becomes available

```
1 emptySlots.P();
2 mutex.P();
3 enqueue x into Q;
4 mutex.V();
5 fullSlots.V();
6 return result;
```

Algorithm 3: Consumer Operation for a Bounded Buffer

Input: Queue Q, semaphores $emptySlots$, $fullSlots$, and $mutex$
Output: One item is removed once data becomes available

```
1 fullSlots.P();
2 mutex.P();
3 remove one item from Q;
4 mutex.V();
5 emptySlots.V();
6 return result;
```

The order of the `P()` operations is crucial. If a producer acquires `mutex` *before* waiting on `emptySlots`, then a full buffer can deadlock the system: the producer sleeps while still holding the mutex, thereby preventing any consumer from entering the buffer code that would free space. By contrast, the order of the `V()` operations usually affects efficiency rather than correctness. Releasing the mutex before waking a peer often avoids waking a thread only to force it to block immediately again.

7.5 Why semaphores are a major step forward

Semaphores solve the most serious weakness of busy waiting: a thread can sleep instead of consuming CPU while waiting for a condition to change. They are also expressive enough to encode both mutual exclusion and event ordering. But they have a cost: the same abstraction is now being used for two rather different purposes, and the programmer must remember by convention which semaphore means what. As programs grow, that ambiguity can make proofs and code reviews harder than necessary.

8 Monitors and Condition Variables

8.1 Why semaphores are not the final abstraction

Semaphores are powerful, but they mix two concepts:

- **mutual exclusion**, which is about protecting shared data;
- **scheduling constraints**, which are about waiting until a predicate becomes true.

In a bounded buffer, for example, one wants a single clean boundary around operations on the queue, yet one also wants to block *inside* those operations when the queue is empty or full. It is a cleaner organization of these responsibilities.

Definition

A **monitor** is a synchronization abstraction consisting of shared data, an associated lock that enforces mutual exclusion on operations over that data, and zero or more condition variables used to wait for predicates over the shared state.

8.2 Condition variables

A condition variable is best understood as a queue of waiting threads associated with some predicate over shared data. It supports three standard operations:

- `Wait(&lock)`: atomically release the lock and sleep; before returning, reacquire the lock;
- `Signal()`: wake one waiting thread, if any;
- `Broadcast()`: wake all waiting threads.

The atomicity of `Wait(&lock)` is the critical innovation. If releasing the lock and going to sleep were separate actions, a wakeup could be lost in the gap between them, recreating exactly the kind of timing bug that synchronization was supposed to eliminate.

Deep Dive

A condition variable is *not* the condition itself. The actual condition is a predicate over shared state, such as "the queue is nonempty." The condition variable is merely the waiting place used when that predicate is false. This is why the code must always re-check the predicate after waking.

8.3 A monitor-based synchronized queue

A simple infinite synchronized queue can be expressed naturally as a monitor:

Listing 7: Monitor-style blocking dequeue.

```
lock.Acquire();
while (queue.isEmpty()) {
    dataReady.Wait(&lock);
}
```

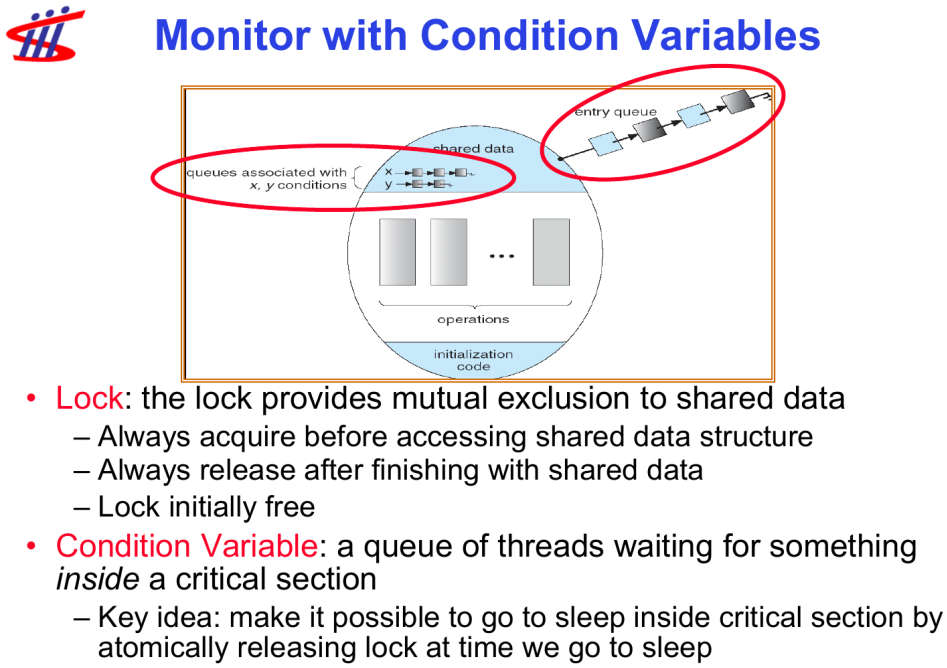


Figure 7: The monitor abstraction as an object combining shared data, monitor operations, an entry queue, and one or more condition-variable wait queues. The key systems idea is that a thread may sleep while conceptually inside the monitor because the `wait` operation atomically releases the lock as it blocks.

```
5 item = queue.dequeue();
6 lock.Release();
7 return item;
```

The corresponding enqueue operation acquires the same lock, inserts an item, signals `dataReady`, and releases the lock. The structure is conceptually clean:

1. entering the monitor acquires mutual exclusion;
2. if the predicate is false, the thread sleeps and temporarily gives up the lock;
3. when awakened, it reenters the monitor and rechecks the predicate;
4. once the predicate holds, it completes the operation.

Many programming environments adopt this pattern directly. Java's built-in monitor model, POSIX condition variables, and numerous language runtimes are all variations on the same fundamental design.

8.4 Hoare monitors and Mesa monitors

We distinguishes two monitor semantics.

Hoare semantics. When thread A signals a condition variable, the waiting thread B immediately acquires the lock and runs. The signaler yields control of the monitor at once. Because

the waiting thread runs immediately, an `if` test can be sufficient: the signaled condition is guaranteed to hold at wakeup time.

Mesa semantics. When thread A signals, the waiting thread is merely moved to a ready queue. The signaler keeps the lock until it later releases it. The scheduler then chooses when the awakened thread actually runs. By the time it reenters the monitor, the predicate may have become false again.

Feature	Hoare monitor	Mesa monitor
Effect of `signal()`	Immediate handoff to waiting thread	Waiting thread becomes runnable later
Who holds the lock next?	The awakened waiter	Typically still the signaler until unlock
Condition check style	`if` can be sufficient	`while` is required in practice
Common setting	Classical textbooks and theory	Most real operating systems and language runtimes

Table 3: Hoare and Mesa monitors differ in the semantics of wakeup and lock transfer.

8.5 Why Mesa monitors require `while`

One example can show exactly why `while` is necessary under Mesa semantics. Suppose thread T_1 is waiting in `RemoveFromQueue()` because the queue is empty. Then thread T_2 enqueues one item and signals the condition variable. Under Mesa semantics, T_1 is only moved to the ready queue; it does not run immediately. Before T_1 gets the CPU, another consumer T_3 may run first, acquire the lock, and remove the sole queued item. When T_1 finally resumes, the queue is empty again.

This is why the correct pattern is not

```
if (queue.isEmpty()) {
    dataReady.Wait(&lock);
}
```

but rather

```
while (queue.isEmpty()) {
    dataReady.Wait(&lock);
}
```

The `while` loop expresses the right semantic meaning: waking from the condition variable does *not* mean that the desired state now certainly holds. It means only that the thread has another opportunity to reacquire the lock and test the predicate again.

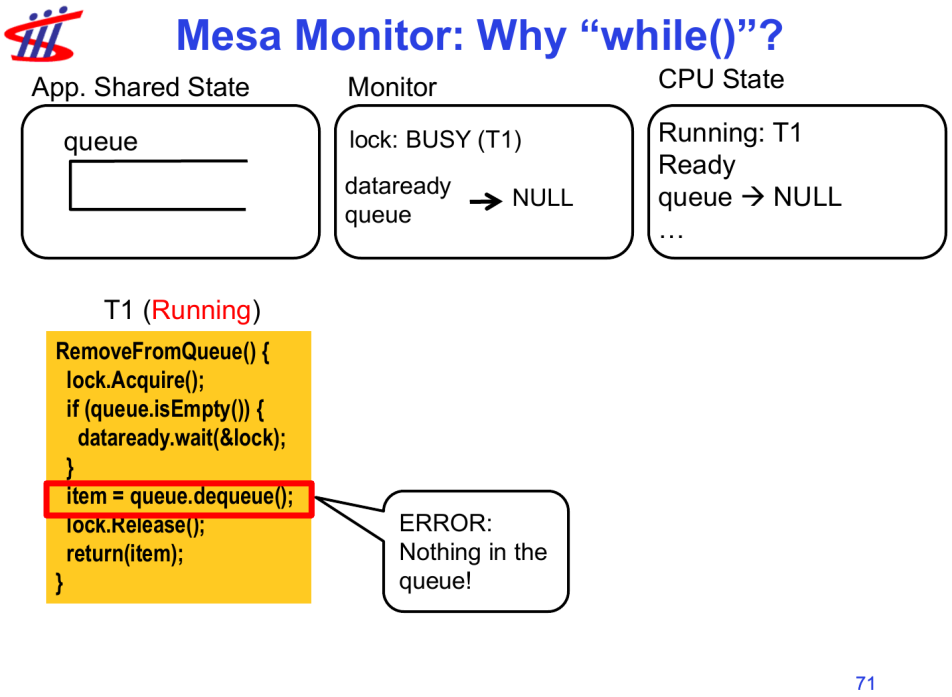


Figure 8: The error state produced when Mesa-style condition variables are paired with `if` rather than `while`. A waiting consumer wakes, assumes work is available, and attempts to dequeue from an empty queue because another thread consumed the only item before the awakened thread reentered the monitor.

Deep Dive

Modern systems programming adds an additional reason to use `while`: some condition-variable implementations permit *spurious wakeups*, meaning that a thread may return from `wait()` even without a matching logical event. Rechecking the predicate in a loop makes the code robust both to Mesa scheduling and to such implementation details.

9 System Design Lessons and Historical Notes

Concurrency mechanisms are not merely syntax. They are design devices that constrain the set of possible executions of a program. Good synchronization makes correctness arguments *simpler*; bad synchronization makes them opaque.

This is why the historical figures associated with the material matter. Dijkstra gave the field a foundational primitive in the semaphore. Hoare formalized monitors and emphasized clarity of reasoning about programs. Lampson and the Xerox PARC tradition helped turn elegant abstract ideas into the Mesa-style semantics that fit real operating systems and schedulers. The relationship between theory and systems practice is visible here in a very concrete way: theory tells us what it means for a concurrent program to be correct, and systems engineering determines how to realize those guarantees efficiently on actual hardware and kernels.

A final design principle is worth stating plainly: a synchronization strategy that is barely understandable at two threads is unlikely to remain trustworthy at twenty. When evaluating a design, one should ask not only "Can this be made to work?" but also "Can I explain why it

works under all interleavings that the operating system allows?"

10 Summary

This lecture develops a layered understanding of concurrency.

First, shell command composition and build systems remind us that systems programming is about explicit control over dependencies, status, and ordering. Second, processes and threads introduce multiple execution contexts, with threads offering cheaper sharing but also exposing programs to races on shared state. Third, race conditions show why concurrent programs must be correct for all interleavings, not merely common ones, and why debugging timing-dependent failures is so difficult.

We then build a hierarchy of synchronization abstractions. Atomic operations provide the foundation. Locks create critical sections and enforce mutual exclusion, but their granularity must be chosen carefully. The "too much milk" problem demonstrates that naive protocols built only from loads and stores are fragile, difficult to prove correct, and often inefficient. Semaphores improve the situation by supporting sleeping wait and by expressing both resource counts and event ordering. Monitors and condition variables go further by separating mutual exclusion from waiting predicates, leading to cleaner and more maintainable code.

The final practical rule is one of the most important in everyday systems programming: when waiting on a condition variable under Mesa-style semantics, always use a `while` loop that rechecks the predicate. The awakened thread has been given another chance to see whether it may proceed; it has not been promised that the desired state still holds.

11 Quizzes & Exercises

11.1 Short-answer questions

1. What is the functional difference between the shell operators `&&` and `||` when running multiple commands?
2. How does building software differ from compiling it?
3. In a Makefile, what is the specific difference between the assignment operators `:=` and `=`?
4. What is the practical "reality of multiprocessing" on a single-core processor?
5. What defines a thread as an execution context, and how does a thread context switch differ from a process context switch?
6. What are three major advantages of allowing threads to cooperate on a task?
7. What is a thread pool, and why is it widely used in web server architecture?
8. What were the software causes of the fatal failures in the Therac-25 radiation therapy machine?
9. Define an atomic operation and explain why atomicity is a necessary building block for synchronization.
10. Explain the trade-off between coarse-grain and fine-grain sharing in terms of implementation complexity and performance.

11.2 Essay prompts

1. Explain why cooperating threads are inherently non-deterministic and often hard to reproduce. Use the "too much milk" problem and the Therac-25 case study to illustrate the dangers of arbitrary instruction interleaving.
2. Compare shared-memory communication and message passing as mechanisms for collaboration among processes or threads. What does each model make easier, and what new problems does each introduce?
3. Describe the essential components of a Makefile, including targets, dependencies, patterns, stems, and a `clean` target. Why does Makefile syntax require such careful attention to formatting?
4. Analyze the progression of attempted solutions to the "too much milk" problem. Why do early note-based solutions fail, and what do those failures reveal about the need for stronger primitives than atomic loads and stores?
5. Evaluate the cost of thread creation and context switching. Why are threads typically cheaper than processes, and why does that fact motivate thread pools in high-traffic servers?

11.3 Reasoning exercises

1. In the bounded-buffer solution, show explicitly why swapping the order of `emptySlots.P()` and `mutex.P()` in the producer can deadlock the system.
2. Write a brief proof argument that a monitor-based blocking dequeue that waits in a `while`-guarded loop is safe under Mesa semantics.
3. Give one example of a system in which message passing would be preferable to shared memory, and one example where shared memory would likely be preferable.
4. Suppose a condition variable is signaled when no thread is waiting. Contrast what happens with a semaphore initialized to 0 in the analogous situation.
5. Consider a UI program that performs network access on the main thread. Explain, in systems terms, why the interface may feel frozen even if the machine is not otherwise overloaded.